

Informe de Benchmarking — Conteo de Frecuencias

Alumno: Daniel Palma - 21524736822

Curso: Programación Avanzada — ICC708

Fecha: 2026-03-25

1. Descripción del problema

Dado un arreglo de n enteros, se desea construir un mapa que asocie cada elemento único con el número de veces que aparece en el arreglo. Por ejemplo:

Input: [1, 2, 3, 1, 2, 1]

Output: {1 → 3, 2 → 2, 3 → 1}

Se comparan dos estrategias con complejidades asintóticas distintas para resolver el mismo problema.

2. Implementaciones

LinearCount — Búsqueda lineal ($O(n^2)$)

Por cada elemento único del arreglo, se recorre el arreglo completo contando sus ocurrencias. Se utiliza un mapa auxiliar para evitar recontar elementos ya procesados, pero el bucle interno se ejecuta una vez completa por cada elemento único.

En el peor caso (todos los elementos son únicos) se realizan $n \times n = n^2$ comparaciones.

HashCount — HashMap ($O(n)$)

En una única pasada sobre el arreglo se incrementa el contador del elemento actual directamente en un HashMap. La operación de inserción/actualización es $O(1)$ amortizado, lo que resulta en un tiempo total $O(n)$.

3. Resultados

Los datos de entrada se generaron con enteros aleatorios en el rango $[0, n/2)$ usando semilla fija (42) para reproducibilidad. Se ejecutaron 5 iteraciones de calentamiento (warm-up) y 20 repeticiones medidas por tamaño. Se reporta la **mediana** de los tiempos en milisegundos.

n	LinearCount (ms)	HashCount (ms)	Ratio L/H
100	0,088	0,056	1,6×
500	0,400	0,138	2,9×
1 000	0,489	0,190	2,6×
5 000	3,688	0,219	16,8×
10 000	14,306	0,250	57,2×
30 000	128,574	1,325	97,0×
50 000	341,374	1,979	172,5×

Mediciones realizadas con OpenJDK 21.0.6, semilla 42, distribución uniforme en $[0, n/2)$.

4. Conclusión

Los resultados confirman el comportamiento esperado por la teoría de complejidad:

- **LinearCount** exhibe crecimiento cuadrático: al triplicar n de 10 000 a 30 000, el tiempo se multiplica por ~ 9 ($3^2 = 9$), exactamente lo predicho por $O(n^2)$.
- **HashCount** crece de forma casi lineal: permanece por debajo de 2 ms en todo el rango medido, con un salto menor alrededor de $n = 30\,000$ atribuible a un redimensionamiento interno del `HashMap`.

Contrario a la expectativa teórica, **no se observó ningún punto de cruce** en el rango medido: `HashCount` fue más rápido incluso en $n = 100$ (0,056 ms vs 0,088 ms). El overhead del hashing en esta JVM resulta menor que el costo de los accesos al arreglo en el bucle interno de `LinearCount`.

HashCount conviene en todos los casos prácticos. La ventaja es de $1,6\times$ para $n = 100$ y crece hasta $172,5\times$ para $n = 50\,000$.

LinearCount solo tiene sentido como referencia didáctica para ilustrar el impacto de la complejidad algorítmica, no como solución de producción.

5. ¿Por qué ambas implementaciones usan HashMap como resultado?

A primera vista puede parecer contradictorio que `LinearCount` — la implementación "sin hash" — use un `HashMap` para devolver su resultado. La razón es triple:

Interfaz común. Ambas clases implementan `FrequencyCounter`, que declara `Map<Integer, Integer> count(int[])`. Esto permite tratar las dos implementaciones de forma intercambiable sin que el código cliente conozca los detalles internos de cada una. Para cumplir el contrato de la interfaz, ambas deben devolver un `Map`. Esto también permite buenas prácticas en la testeabilidad.

Buenas prácticas de diseño. El tipo de retorno declarado es la interfaz `Map`, no la clase concreta `HashMap`. Esto significa que en el futuro se podría cambiar la implementación interna del resultado (por ejemplo, a `TreeMap` para obtener claves ordenadas) sin modificar ningún cliente. Usar `HashMap` como implementación concreta del resultado es una decisión de eficiencia, no un acoplamiento.

No afecta la comparación. La diferencia de rendimiento entre ambos algoritmos reside en cómo se *obtiene* cada conteo, no en cómo se *almacena* el resultado final. `LinearCount` usa el `HashMap` solo para guardar los totales ya calculados mediante el bucle doble; `HashCount` lo usa para calcular y almacenar en una misma pasada. El costo de construir el `HashMap` de resultado es $O(k)$ en ambos casos (siendo k el número de elementos únicos) y, por tanto, no altera la diferencia asintótica entre $O(n^2)$ y $O(n)$.

6. Limitaciones

- **Ruido de JVM:** el recolector de basura (GC) puede pausar la ejecución durante una medición, inflando ese sample. El uso de la mediana mitiga este efecto, pero no lo elimina completamente.
- **Calentamiento del JIT:** se usaron 5 iteraciones de warm-up. En algunos escenarios el compilador JIT puede requerir más iteraciones para optimizar completamente el código, especialmente en

tamaños pequeños.

- **Tamaños de entrada:** los resultados son válidos para el rango medido (100–50 000). Extrapolar hacia valores mucho mayores o menores requiere mediciones adicionales.
- **Distribución del input:** se usó distribución uniforme aleatoria con rango $n/2$. Distribuciones distintas (todos iguales, todos únicos, datos ordenados) pueden afectar el comportamiento del HashMap (colisiones) o del cache de CPU, alterando los tiempos relativos.
- **Entorno:** los tiempos son específicos al hardware y JVM utilizados. Los resultados pueden diferir en otras máquinas.